

Modelo de predicción de masa de pingüinos

Autora: Andrea Medina Rico

Matrícula: A01705541

Abstracto. Este reporte detalla la creación e implementación de un modelo para predecir la masa en kilogramos de un pingüino, de acuerdo a algunas de sus características. Para esto, el modelo fue entrenado y evaluado con una regresión lineal múltiple, de la mano con un algoritmo para su optimización. Este modelo reporta un coeficiente de determinación de 0.814, indicando la capacidad de explicar el 81.4% en la variación de la masa de los pingüinos.

I. INTRODUCCIÓN

En el ámbito de la zoología, un modelo de predicción de la masa de un animal a partir de algunas de sus medidas corporales puede resultar una gran herramienta. Esto reduciría la necesidad de procesos invasivos y costosos para mantener registro de sus medidas y su salud.

El objetivo de este modelo es generar una predicción de la masa de un pingüino a partir de algunas de sus medidas y sus características. El conjunto de datos 'penguins_size.csv', presenta información de pingüinos de tres distintas especies dentro de la misma familia. Se busca demostrar que se puede realizar la predicción de una característica corporal (en este caso, la masa), sin importar que un animal pueda tener variaciones dependiendo de la raza o especie que sea.

II. EXTRACCIÓN, TRANSFORMACIÓN Y LIMPIEZA DE DATOS (ETL)

a. Descripción del conjunto de datos

La información de los pingüinos proviene del archivo 'penguins_size.csv'. Contiene información sobre pingüinos de tres islas distintas: Dream, Biscoe y Torgersen, las tres situadas en la Antártida. Además, distingue entre las tres especies de la familia Pygoscelis, conocida como "pingüinos de cola de cepillo". Estas son

Adelia (Adelie), Gentú o Papúa (Gentoo) y Barbijo (Chinstrap).

Variables numéricas:

- *culmen_depth_mm*: Alto de la cresta superior del pico (culmen) medida en milímetros.
- *culmen_length_mm*: Longitud de la cresta superior del pico (culmen) medida en milímetros.
- *flipper_length_mm*: Longitud de la aleta medida en milímetros.
- *body_mass_g*: Masa del pingüino medida en gramos.

Variables categóricas:

- *sex*: Sexo (macho o hembra).
- *species*: Especie del pingüino.
- *island*: Isla de proveniencia.

b. Transformación de los datos

Primeramente, se eliminaron los ejemplos con datos faltantes o nulos, debido a que eran 11, una cantidad baja.

Las variables categóricas, al ser de clasificación en lugar de numéricas, no pueden ser directamente procesadas por el modelo. Por ello, se procesaron de dos maneras distintas.

La columna hembra podía tener dos valores distintos, 'male' o 'female'. Se le dio un valor binario donde todos los machos corresponden a 1 y las hembras a 0.

En el caso de la especie y la isla, ambas columnas pueden tener tres valores distintos. Una transformación numérica como en el caso anterior no es lo más ideal, debido a que el modelo comienza a asociarlo con una relación lineal, donde una categoría es “mayor o menor” que otra. Por ello, se aplicó one-hot encoding. Se crearon $k - 1$ nuevas columnas binarias, siendo k el número de valores únicos en la columna a dividir. Estas nuevas columnas se conocen como “variables dummy”.

Al generar $k - 1$ columnas, se evita caer en la trampa donde dos de las variables dummy tienen una colinealidad perfecta. Se toma como base la categoría no incluida (si todas las columnas son 0, corresponde a la categoría base). Se añadieron *species Adelie*, *species Gentoo*, *island Dream* e *island Biscoe*.

En las transformaciones numéricas, se realizó una conversión de unidades para que la diferencia entre las categorías no fuera tan grande. Los gramos pasaron a kilogramos y los milímetros a centímetros.

Con esto, las columnas también fueron renombradas acorde con el cambio.

c. Detección de anomalías

Para observar el comportamiento de cada variable numérica, se realizó un histograma por cada una. Esto demostró que no existía ningún valor atípico que sesgara los datos, pero sí demostraron la tendencia de los datos a repartirse en dos grupos.

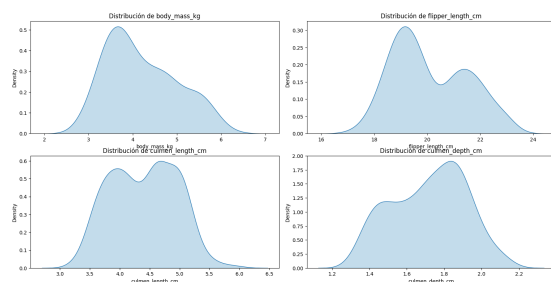


Figura 1. Distribución de variables numéricas

La distribución de los datos se aglomera tomando como media dos puntos distintos. Esto podría deberse a un solapamiento entre las mismas especies al ser estas parte de la misma familia. Por ejemplo, en una categoría, dos especies podrían presentar una característica similar, mientras que la tercera tiene valores distintos.

III. PREPROCESAMIENTO DE LOS DATOS

a. Elección de variables

Para mayor entendimiento sobre cómo se relaciona la masa de los pingüinos con el resto de las categorías, se realizó una matriz de correlación. En cada celda, muestra un número entre -1 y 1, donde -1 es una fuerte relación negativa y 1, una fuerte relación positiva.

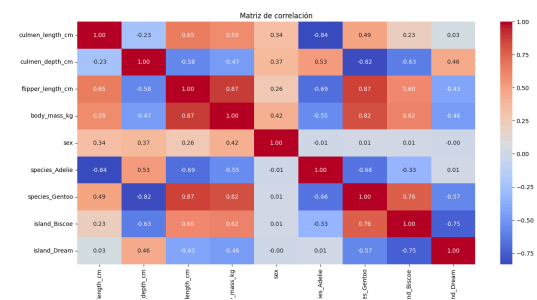


Figura 2. Matriz de correlación entre variables

En la fila o columna de *body_mass_kg*, se observa que, en general, tiene relaciones fuertes y medianas con la mayoría de las variables. Al no haber ninguna con correlación baja ni que se considere no importante para la predicción, se decidió conservarlas todas.

La relación entre la variable a predecir (la masa) contra cada una de las categorías se visualiza en la siguiente figura:

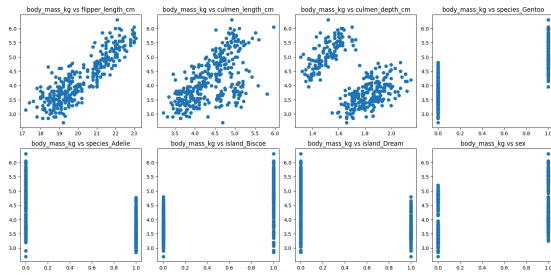


Figura 3. Diagramas de dispersión de las categorías selectas contra la masa en kg.

Se observa una relación cercana a lineal en las tres variables numéricas. El comportamiento de las demás gráficas se debe a que son los valores binarios (0 y 1) provenientes de las variables categóricas transformadas (*sex*, *species* e *island*). A pesar de no ser tan claro como en las numéricas, se observa una clara distinción de los valores de la masa cuando pertenecen o no a una categoría, es decir, cuando su valor es 0 o 1.

b. División de los datos

Una manera de asegurar que el modelo aprendió adecuadamente y sus resultados son acertados, es tener tres conjuntos de datos: entrenamiento (train), validación (validation) y pruebas (test). Así, el modelo aprende de datos distintos a los que se utilizan para su evaluación y no se genera una idea falsa del desempeño del mismo, sino que se realiza un diagnóstico adecuado.

En este contexto, tenemos un total de 333 instancias tras la limpieza. Con el fin de tener la mayor cantidad posible a destinar para entrenamiento, el conjunto se divide en 80% train y 20% pruebas, siendo 266 y 67 registros, respectivamente.

Posteriormente, se utiliza el algoritmo cross validation para la etapa intermedia de validación, debido a la limitada cantidad de instancias. Este funciona con el conjunto de datos de train, por lo que no es necesaria realizar otra división adicional.

IV. MODELO DE REGRESIÓN LINEAL

a. Regresión lineal múltiple

Este modelo sigue una regresión lineal múltiple para explicar la relación entre la variable a predecir (la masa del pingüino) respecto a las variables independientes seleccionadas.

$$y = \beta_0 + \sum_{i=1} \beta_i x_i$$

Figura 4. Función de hipótesis para regresión lineal múltiple.

La función de hipótesis es la ecuación lineal que describe el peso de cada variable para llegar al resultado final. El resultado de esta ecuación es la predicción de la variable dependiente.

b. Función de costo

La función de costo, también conocida como función de pérdida o función de error, compara el valor predicho por la función de hipótesis contra el valor real de nuestra variable a predecir (en este caso, la masa del pingüino). Como su nombre lo dice, mide el error o la pérdida que hubo. Este valor es de suma importancia, debido a que nos ayuda a evaluar el desempeño del modelo y saber qué tan cerca estamos matemáticamente de realizar una predicción acertada. El objetivo de este modelo será acercarlo a 0, indicando que la mayoría de los valores son acertados o tienen poco rango de error. Se utilizan las dos ecuaciones siguientes:

$$MSE = \frac{\sum (y_i - p_i)^2}{n} \quad MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Figura 6. Funciones para cálculo del error.

La primera (MSE) se utiliza en el algoritmo de optimización, para calcular el cambio en los parámetros necesario para minimizar el error. La segunda (MAE), se utiliza como error promedio interpretable en las unidades de medida de nuestra variable a predecir, en este caso, en kilogramos.

c. Gradiente descendiente

La manera en la que el modelo minimizará su error es utilizando el algoritmo de optimización gradiente descendiente (gradient descent) para su entrenamiento. Este algoritmo encuentra los parámetros de la función de hipótesis necesarios para minimizar lo más posible la función de costo. Se define como:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

Figura 7. Función para calcular el nuevo valor de un parámetro con gradiente descendiente.

Para calcular el valor de un nuevo parámetro de la función de hipótesis (la ecuación lineal múltiple), a su valor actual se le resta el gradiente de la función de costo. Desarrollada para una regresión lineal múltiple, queda de la siguiente forma:

$$\Theta_j := \Theta_j - \frac{\alpha}{m} \sum_{i=1}^m [(h_{\Theta}(x_i) - y)x_i]$$

Figura 8. Función del gradiente descendiente desarrollada para regresión lineal.

La optimización se apoya de dos hiper parámetros: el número de épocas y el learning rate (alpha) y el número de épocas. El número de épocas es la cantidad de veces que se realizará la optimización de parámetros. Una época se da por concluida cuando todos los parámetros han sido optimizados y, hasta entonces, se actualizan todos los valores.

El learning rate o tasa de aprendizaje es el tamaño de los pasos que el algoritmo da hacia el gradiente. Define el grado en el que el algoritmo de optimización ajusta sus parámetros. Su valor es óptimo cuando se encuentra en balance, pues un valor demasiado bajo puede ocasionar lentitud en el aprendizaje sin un avance significativo. Por otro lado, un valor demasiado alto evita que se alcance la convergencia.

d. Validación cruzada

El algoritmo de validación cruzada (cross validation) se utiliza para probar la eficiencia del modelo, con el objetivo de ajustar los hiper parámetros y mejorarlo. Se utiliza en este modelo debido a las pocas instancias en el conjunto de datos.

Del 80% de datos destinados a entrenamiento, se divide en k subpartes (en este caso 10). En cada iteración, se toman 9 partes para entrenar y una para validar, repitiendo hasta usar todos los datos para validación. Esto permite evaluar el error arrojado y ajustar los hiper parámetros, como el número de épocas a repetir el algoritmo de optimización o el learning rate (el tamaño de los pasos al optimizar).

e. Normalización de datos

La normalización de los datos es una técnica de escalamiento, la cual estandariza el rango de valores de cada característica. Esto evita que el modelo se sesgue a otorgarle mayor peso a las variables de valores más altos. Permite que cada variable tenga una contribución proporcional a la ecuación.

En este modelo, utilizaremos la estandarización o z-score, debido a que es comúnmente utilizado en regresiones lineales y no resulta tan afectado por valores extremos. Resulta favorable considerando la distribución de las

categorías, aglomerada en dos grupos y con un rango de valores considerable. Está dada por la ecuación:

$$Z = \frac{X - \mu}{\sigma}$$

Figura 9. Función de estandarización de datos.

La estandarización se realiza tomando la media y desviación estándar por categoría del conjunto de datos destinados para entrenamiento. Se aplica a todas las categorías tanto de train como de test, a excepción de los valores predichos y por predecir.

f. Evaluación del modelo

Los resultados del modelo se evaluarán con el coeficiente de determinación R^2 , un valor entre 0 y 1 que indica cuántos datos es capaz de explicar el modelo. Al multiplicarse por 100, es interpretable para nosotros como porcentaje. Se calcula como:

$$R^2 = \frac{\sum_{t=1}^T (\hat{Y}_t - \bar{Y})^2}{\sum_{t=1}^T (Y_t - \bar{Y})^2}$$

Figura 10. Función del coeficiente de determinación.

g. Criterios de diagnóstico

Las fallas en los modelos de predicción pueden deberse a dos factores: el sesgo y la varianza.

- *Underfitting*: presenta un alto sesgo, por lo que la cantidad de errores es alta tanto en el entrenamiento como en las pruebas, indicando que no logra captar el patrón de los datos.
- *Overfitting*: debido a su alta varianza, la cantidad de errores en entrenamiento es mínima, pero

aumenta considerablemente en las pruebas. El modelo memoriza antes de aprender patrones.

- *Fitting*: el modelo tiene buena generalización, lo que indica que ambos varianza y sesgo son altos. Tiene la capacidad de entender y aplicar los patrones aprendidos en datos no conocidos previamente.

El análisis de las gráficas de error y distribución de residuales aportan una clara visión del diagnóstico en el que cae el modelo. En las curvas del error por épocas, se busca que las tres etapas (train, validation, test) sean consistentes y cercanas entre sí. En la distribución del error, se busca que esté centrada en el cero, con una distribución normal y anchura reducida. Ambos indicarían sesgo y varianza bajos.

V. EVALUACIÓN INICIAL

a. Resultados iniciales

Los datos de train se utilizaron en la validación cruzada con una constante $k = 10$. Así, por cada iteración se mantuvo el 90% para entrenamiento y 10% para evaluación. En cada iteración, se estandarizaron los datos tomando la media y desviación estándar del respectivo conjunto de train. Para cada iteración, se obtuvo la pérdida por épocas. Posteriormente, con la media y desviación estándar del conjunto completo de train, se estandarizaron los valores predictores de train y test.

Se realizaron dos iteraciones, con 3000 y 5000 épocas, con el fin de comparar ambos resultados y evaluar la significancia de agregar más épocas al entrenamiento. El learning rate para ambos se estableció en 0.001.

Epoch		MSE	MAE	R2
3000	Train	0.065	0.292	0.796
	Val	1.530	1.344	0.779
	Test	0.084	0.344	0.748
5000	Train	0.041	0.229	0.870
	Val	0.938	0.906	0.855
	Test	0.062	0.290	0.814

Tabla 1. Pérdida y precisión en las tres etapas de entrenamiento con dos configuraciones.

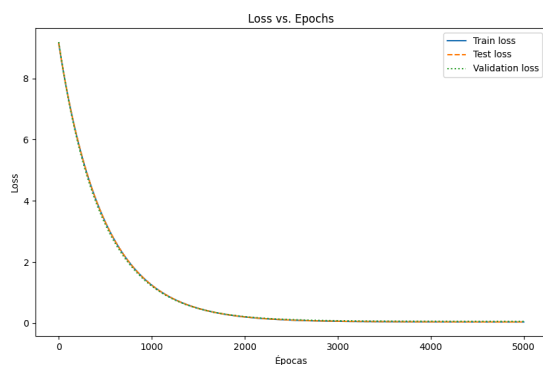


Figura 13. Pérdida en las tres etapas de entrenamiento del modelo con 5000 épocas.

Si bien el modelo ejecutado con 5000 épocas presenta mejores resultados, la diferencia no es tan remarcable, así como lo muestra la gráfica, indicador de que se encuentra cerca de la convergencia a partir de la época 3000. Se opta por la opción de mantener 5000 épocas, sabiendo que cualquier número mayor no tendría una mejora impactante.

b. Diagnóstico

Se resaltan los siguientes puntos de acuerdo al comportamiento del error:

- El rango de errores vistos va de 0 a 6 ocurrencias (frecuencia).
- La distribución de residuales no es normal, pues tiene dos puntos altos de mayor frecuencia.

- La mayor cantidad de errores tienen una magnitud alrededor de 0.2 kg, de acuerdo a la unidad de medida de la variable a predecir. La segunda magnitud más repetida es de -0.4 kg.
- Si bien la cantidad de errores nulos o mínimos es alta, no es la más repetida.
- La desviación estándar de los errores es de 0.35.

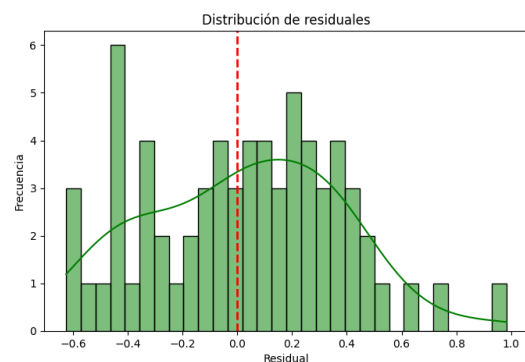


Figura 13. Análisis de distribución de residuales

• Sesgo:

Se observa que los errores son cercanos a 0. A nivel general, el modelo es capaz de replicar el patrón de los datos. La distribución indica que el modelo tiene un **s sesgo medio**, pues el promedio de los residuales está levemente alejada de 0 y hay una mayor frecuencia en 0.2 y -0.4, valores alejados. Aunque no acierta el valor con exactitud, se aproxima consistentemente.

En cuanto al comportamiento de la pérdida, la comparación entre entrenamiento y validación demuestra resultados casi idénticos. En ambos casos, se acerca a errores casi nulos, por lo que el modelo generado no tiene un sesgo alto y no presenta resultados completamente desajustados a lo esperado. Es decir, el modelo no cae en un escenario de subajuste (*underfitting*).

- **Varianza:**

Los residuales presentan una desviación estándar de 0.35, interpretable como que, si bien existe una variabilidad en los datos, siguen estando en su mayoría cercanos a 0. El modelo tiene **varianza media**, pues si bien mantiene sus errores cercanos a cero, existe variabilidad en sus valores. Los residuales no tienden a disminuir conforme se alejan del centro, reflejando varianza en el ancho de la curva. Los errores más grandes son bastante comunes.

Además, los resultados indican que el modelo no se sobreajustó en el entrenamiento tal que su desempeño de evaluación resultara opuesto al de train debido a la similaridad de resultados en distintos conjuntos de datos. Es decir, no cayó en un escenario de *overfitting*.

El comportamiento de pérdida es bastante similar entre las tres etapas (*train*, *validation*, *test*) tal que las curvas se solapan. Por consiguiente, se indica que el modelo es capaz de aprender de un conjunto de datos y adaptarse a nuevos sin conocerlos previamente.

c. Predicción

Los parámetros finales del modelo de regresión indican que, en general, las categorías seleccionadas tienen una baja relación lineal con la variable a predecir. Sin embargo, es la unión de todas ellas en conjunto la que garantiza un porcentaje de precisión alto.

Parámetros:

culmen_length_cm = 0.063

culmen_depth_cm = $-7.37 \cdot 10^{-4}$

flipper_length_cm = 0.290

sex = 0.237

species_Adelie = 0.018

species_Gentoo = 0.261

island_Biscoe = 0.107

island_Dream = -0.05

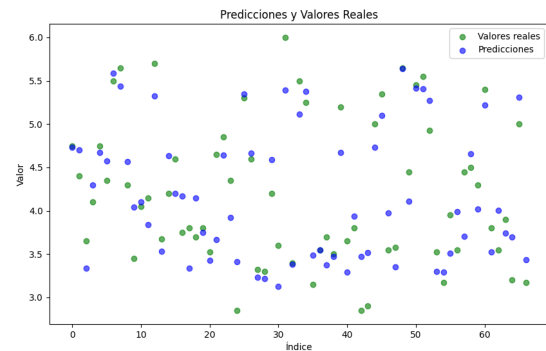


Figura 13. Comparación de valores predichos del modelo contra los valores reales.

En el diagrama de dispersión, se plasma el comportamiento de los valores predichos y reales sobre la masa de los pingüinos. A simple vista, se observa que tienen una varianza y comportamiento similar. Una gran parte de los puntos está sobrepuesto, indicando la predicción correcta, o muy cercano, indicando un error mínimo que no sobrepasa los 200 gramos. En el resto de puntos, aunque son menos, se observa un error mayor. Este caso se presenta más en los pesos reales muy bajos o muy altos, fuera del promedio. Esto puede indicar que al modelo le cuesta adaptarse a valores extremos.

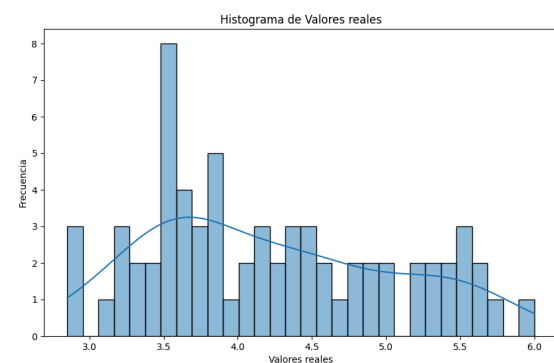


Figura 14. Histograma y distribución de los valores reales de masa (test).

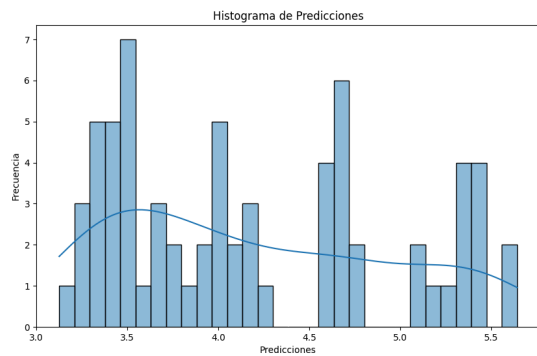


Figura 15. Histograma y distribución de los valores **predichos** de masa (test).

Las gráficas demuestran que la cantidad de valores repetidos no es la misma, existe una varianza evidente. Esto se explica con el coeficiente de determinación, pues hay un 18.6% de datos que nuestro modelo no alcanza a predecir adecuadamente, por lo que el modelo no es completamente preciso. Sin embargo, se observa una distribución bastante similar en ambas. Esto quiere decir que el modelo sí aprende y sigue la tendencia general de los datos.

VI. MODELO CON RANDOM FOREST

Se realizará otro modelo de predicción aplicando Random Forest con la librería de python scikit-learn, buscando predicciones más acertadas y con menor error. Random Forest es un algoritmo que utiliza múltiples árboles de decisión débiles y los une con el método de ensamble Bagging (Bootstrap Aggregating). Los árboles generados no están correlacionados entre sí debido a la aleatoriedad de la muestra de datos y las características empleadas, lo que otorga un resultado estadísticamente significativo.

Se generan diversas muestras de los datos con reemplazo, lo que significa que una instancia individual puede repetirse en diversas muestras. Cada vez que se obtiene una muestra, se selecciona un conjunto aleatorio de características a considerar

para la creación del árbol. Así, el árbol de decisión se genera tomando en cuenta las características seleccionadas, la profundidad máxima y la cantidad máxima de hojas establecidas.

En el contexto de regresión, donde se busca predecir un valor numérico, se promedian los árboles para obtener un modelo capaz de generalizar datos.

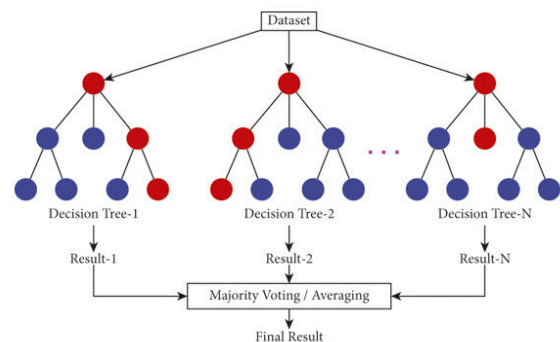


Figura 14. Diagrama explicativo del Random Forest.

Existen diversos hiper parámetros a definir para guiar el desempeño del modelo. Los utilizados en este son:

- **n_estimators:** Cantidad de árboles. Aumentar la cantidad aumenta el desempeño.
- **max_features:** Máximo de características a considerar por árbol.
- **max_depth:** Máximo de niveles por árbol. Un valor bajo conduce a underfitting y un valor alto, a overfitting.
- **max_samples:** Máximo de instancias utilizadas como muestra para los árboles.
- **min_samples_split:** Mínimo de instancias necesarias para que un nodo pueda dividirse en otros.

VII. EVALUACIÓN CON FRAMEWORK

a. Resultados con framework

Al igual que el modelo anterior, los datos de train se utilizaron en la validación cruzada con una constante $k = 10$. Así, por cada iteración se mantuvo el 90% para entrenamiento y 10% para evaluación. En cada iteración, se estandarizaron los datos tomando la media y desviación estándar del respectivo conjunto de train. Para cada iteración, se obtuvo la pérdida final, así como los residuos por cada instancia predicha. Se utilizaron los hiper parámetros por defecto dentro de la librería de scikit learn.

Hiper parámetros:

$n_estimators = 100$

$max_features = sqrt$

$max_depth = None$

$max_samples = None$

$min_samples_split = 2$

$random_state = 42$

	MSE	MAE	R2
Train	0.013	0.093	0.978
Val	0.099	0.252	0.830
Test	0.095	0.248	0.857

Tabla 2. Pérdida en las tres etapas de entrenamiento del modelo con Random Forest.

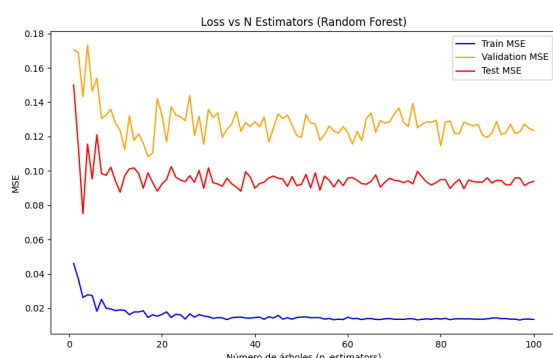


Figura 14. Pérdida entre train, validation y test por cantidad de árboles.

b. Diagnóstico

Se resaltan los siguientes puntos de acuerdo al comportamiento del error:

- El rango de errores vistos va de 0 a 8 ocurrencias (frecuencia).
- La distribución de residuales puede considerarse de *t-student*. Tiende a ser normal, aunque tiene mayor frecuencia de datos en las colas.
- El error con mayor frecuencia es de -0.2 kg. A pesar de que la mayor frecuencia no es precisamente el error con magnitud 0, la distribución está cerca de centrarse en el cero. Hay menos valores en los extremos, generando la curva casi simétrica.
- La desviación estándar de los residuales es de 0.301.

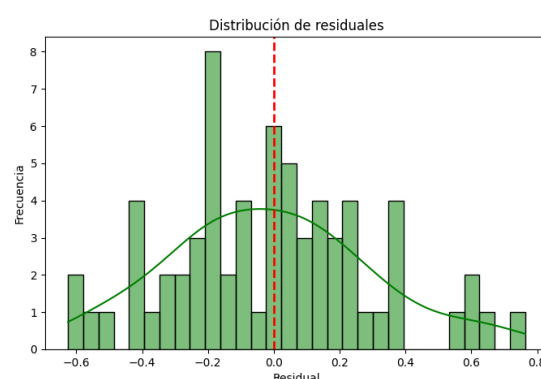


Figura 13. Análisis de distribución de residuales

• Sesgo:

La distribución de los residuales está casi centrada en el cero y disminuye conforme se alejan del mismo. El modelo tiene un **s sesgo bajo**, puesto que, en su mayoría, acierta los valores o presenta un error mínimo. Esto se refuerza con la curva del error, debido a que el error en el entrenamiento es mínimo y muy cercano a 0. Además de un sesgo bajo, implica un descarte de la probabilidad de un subajuste (*underfitting*), pues el modelo sí es capaz de predecir el patrón de los datos.

- **Varianza:**

El modelo presenta una **varianza media**, debido a que tiene un abarcamiento considerable en el rango de errores. Abarca un gran rango de errores, donde los límites no son valores anómalos, sino que siguen teniendo una frecuencia baja pero importante.

En cuanto a la curva del error, se observa que el error presenta picos altos y bajos en la validación y pruebas. Si bien generaliza el patrón de los datos, su desempeño es inestable al exponerse a nueva información. Estos picos pueden ser producto de la cantidad reducida de árboles. El tener menos productos independientes de donde promediar puede generar mayor ruido en los cambios y mayor sensibilidad a los árboles utilizados.

El modelo presenta una tendencia a sobreajustarse en el entrenamiento, es decir, *overfitting*. La diferencia de error entre validation y test es de 0.004, lo que indica que el resultado que el modelo tiene a datos nuevos es similar y replicable. Sin embargo, train tiene un error bastante cercano a 0, mientras que las otras etapas están ligeramente más altas. Indica un ajuste adecuado en los valores con los que fue entrenado y una mayor dificultad para predecir con datos nuevos. Aún con estos resultados, el modelo alcanza una precisión del 85.7% en pruebas, superando el desempeño del modelo anterior.

c. Predicción

Al observar las predicciones contra los valores reales, se identifica que, en efecto, el modelo no tiene un subajuste. A pesar de que no todos los valores son acertados, se observa la tendencia a seguir el mismo patrón de comportamiento.

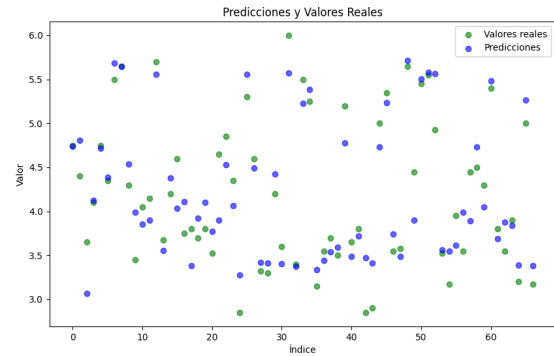


Figura 13. Comparación de valores predichos del modelo contra los valores reales.

VIII. MEJORA DEL MODELO CON FRAMEWORK

a. Selección de hiper parámetros

Con el objetivo de reducir la varianza del modelo y, con ello, reducir su tendencia a *overfitting*, se regularizan los hiper parámetros. Se busca encontrar una combinación de hiper parámetros adecuada, de acuerdo a sus características, para mejorar la precisión del modelo.

- ***n_estimators*:**

Aumentar la cantidad reduce la varianza del modelo, lo que puede contribuir a la prevención del *overfitting*. Dado que el conjunto de datos es relativamente pequeño, 100 árboles son suficientes para dar una predicción precisa. Sin embargo, se aumentará hasta 300 árboles.

- ***max_features*:**

En este caso, se mantiene el valor de *sqrt*, indicando que será la raíz cuadrada del total de categorías que se tenga. Un número pequeño lleva a una menor varianza de predicciones.

- ***max_depth*:**

Está relacionado con *min_samples_split*, puesto que ambos ayudan a limitar el crecimiento de los árboles. Se busca mantenerlos pequeños para cumplir con el principio de random forest: reducir la

varianza del modelo con el promedio de muchos árboles débiles. Por esta razón, se optó por un valor de 15

- **max_samples:**

En este contexto, el número de instancias utilizadas resulta relevante para la precisión del modelo. Durante la validación de la regresión lineal, se observó la diferencia de entrenar con el 80% de train en *cross validation* contra el conjunto completo destinado a train. Por ello, se decidió utilizar el 80% de todos los datos, manteniendo el tamaño de la división de train.

- **min_samples_split:**

Lo más adecuado es mantenerlo bajo por la baja cantidad de datos. Sin embargo, se aumentará a 4 para limitar la facilidad de cada árbol a continuar creciendo y que la profundidad sea significativa.

b. Resultados finales

Fue utilizado el mismo proceso de *cross validation* y estandarización de datos que en las iteraciones anteriores.

Hiper parámetros:

n_estimators = 300
max_features = sqrt
max_depth = 15
max_samples = 0.8
min_samples_split = 4
random_state = 42

	MSE	MAE	R2
Train	0.030	0.135	0.953
Val	0.087	0.232	0.850
Test	0.088	0.237	0.866

Tabla 3. Pérdida en las tres etapas de entrenamiento con hiper parámetros mejorados.

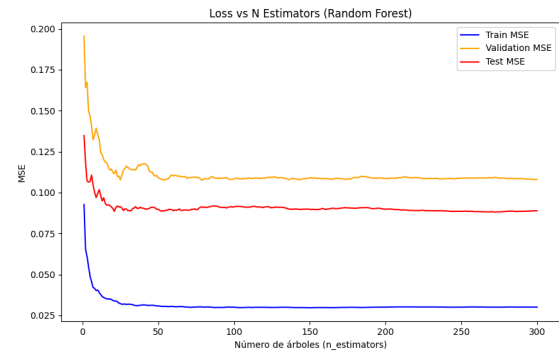


Figura 14. Pérdida entre train, validation y test por cantidad de árboles.

c. Diagnóstico

Se resaltan los siguientes puntos de acuerdo al comportamiento del error:

- El rango de errores vistos va de 0 a 7 ocurrencias (frecuencia).
- La distribución de residuales puede considerarse de *t-student*. Tiende a ser normal, aunque tiene mayor frecuencia de datos en las colas. Una de ellas ya es mínima.
- El error con mayor frecuencia es de -0.2 kg, seguido de 0.2 y 0. A pesar de que la mayor frecuencia no es precisamente el error con magnitud 0, la distribución está cerca de centrarse en el cero. Hay menos valores en los extremos, generando la curva casi simétrica.
- La desviación estándar de los residuales es de 0.298.

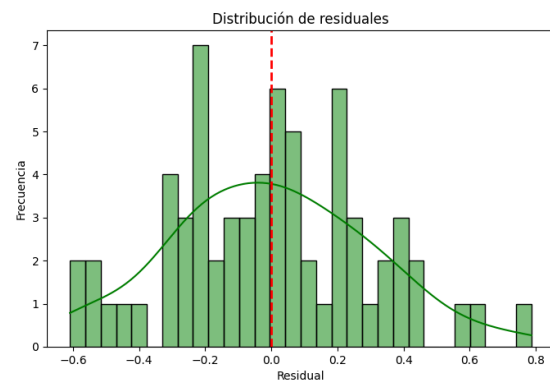


Figura 13. Análisis de distribución de residuales

- **Sesgo:**

La distribución de los residuales, al igual que el modelo anterior, está casi centrada en el cero y disminuye conforme se alejan del mismo. El modelo mantiene un **sesgo bajo**, puesto que, en su mayoría, acierta los valores o presenta un error mínimo. Los valores más alejados del cero ya tienden a ser menores. A su vez, el error en el entrenamiento es mínimo y muy cercano a 0. Además de un sesgo bajo, implica un descarte de la probabilidad de un subajuste (*underfitting*), pues el modelo sí es capaz de predecir el patrón de los datos.

- **Varianza:**

El modelo mantiene una **varianza media**, aunque mejoró al agrupar una gran frecuencia de datos cercana al cero. Abarca un gran rango de errores, donde los límites no son valores anómalos, sino que siguen teniendo una frecuencia baja, pero algunas de ellas importantes para influir en el diagnóstico.

Comparado con el modelo anterior, existe menor ruido o menos altibajos en las gráficas. Su desempeño se estabilizó y ahora representa una mejor capacidad de adaptarse a la tendencia de los datos. Por otro lado, el conjunto de entrenamiento tiene una evidente diferencia con la validación y pruebas, propiamente debido a ser evaluado con datos ya conocidos. Esto indica que, si bien ha disminuido la varianza, sigue existiendo una tendencia al sobreajuste (*overfitting*), aunque el modelo sea capaz de generalizar patrones.

Existe varianza en el modelo, pero es lo suficientemente baja para no afectar a gran nivel las predicciones. Se concluye que el modelo entiende y aplica los patrones con un 86.6% de precisión en pruebas.

d. Predicción

La mayoría de los valores predichos está en

contacto con los puntos de valores reales o, al menos, cercanos tal que se identifica a simple vista cada par de puntos y el patrón que generan.

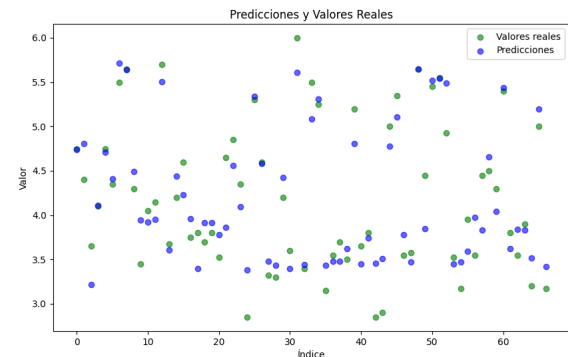


Figura 13. Comparación de valores predichos del modelo contra los valores reales.

IX. CONCLUSIÓN

Los tres modelos generados tuvieron un buen desempeño, teniendo una precisión mayor a 80%. Las mejoras encontradas radican en la complejidad del modelo y los hiper parámetros seleccionados para su entrenamiento. En la siguiente tabla, se realiza un análisis comparativo entre los tres.

Epoch		MSE	MAE	R2
Model 1	Train	0.041	0.229	0.870
	Val	0.938	0.906	0.855
	Test	0.062	0.290	0.814
Model 2	Train	0.013	0.093	0.978
	Val	0.099	0.252	0.830
	Test	0.095	0.248	0.857
Model 3	Train	0.030	0.135	0.953
	Val	0.087	0.232	0.850
	Test	0.088	0.237	0.866

Tabla 1. Comparación de pérdida entre los tres modelos generados.

La diferencia más remarcable es del modelo de regresión lineal múltiple contra los de Random Forest. El coeficiente de determinación aumenta en aproximadamente 5% y el error se reduce en gramos. Este puede resultar un indicador de que el comportamiento de los datos no es completamente ajustable a relaciones lineales. Se comprueba observando los valores finales de los parámetros, puesto que ninguno resultó tener una relación mayor a 0.3.

El modelo con el mejor desempeño fue el tercero, random forest con hiper parámetros ajustados. Comparado con el random forest por defecto, presentó una mejora en el porcentaje de precisión de 0.09%. Sin embargo, el problema de varianza media prevaleció con su tendencia al *overfitting*.

En otras palabras, la regularización de hiper parámetros funcionó para explicar un mayor porcentaje de los datos, pero a nivel diagnóstico se encuentran ambos en la misma posición. Para llegar a considerar los modelos como *fitting*, se tendría que observar una reducción clara en la variabilidad de los errores, manteniéndose todos similares y aglomerados alrededor del cero en distancias mínimas.

X. REFERENCIAS

Bobbitt, Z. (2021, 2 febrero). What is the Dummy Variable Trap? (Definition & Example). Statology.
<https://www.statology.org/dummy-variable-trap/>

3.1. Cross-validation: evaluating estimator performance. (s. f.). Scikit-learn.
https://scikit-learn.org/stable/modules/cross_validation.html

Brownlee, J. (2016, 21 junio). How to choose the right test options when evaluating machine learning algorithms. Machine

Learning Mastery.
<https://machinelearningmastery.com/how-to-choose-the-right-test-options-when-evaluating-machine-learning-algorithms/>

Wagavkar, S. (2024, 20 noviembre). Introduction to the Correlation Matrix. Built In.
<https://builtin.com/data-science/correlation-matrix>

Donges, N. (2024, 1 agosto). Gradient Descent in Machine Learning: A Basic Introduction. Built In.
<https://builtin.com/data-science/gradient-descent>

Validación cruzada y k-fold cross-validation | Codificando Bits. (2024b, febrero 25). Codificando Bits.
<https://codificandobits.com/blog/validacion-cruzada-k-fold-cross-validation/>

Jaiswal, S. (2024, 4 enero). What is normalization in machine learning? A comprehensive guide to data rescaling. Datacamp.
<https://www.datacamp.com/tutorial/normalization-in-machine-learning>

Bergmann, D., & Stryker, C. (2025). Loss Function. IBM.
<https://www.ibm.com/think/topics/loss-function>

Stewart, & Ken. (2025, 19 julio). Mean squared error (MSE) | Definition, Formula, Interpretation, & Facts. Encyclopedia Britannica.
<https://www.britannica.com/science/mean-squared-error>

Siddiqui, L. (2025, 8 julio). Coeficiente de determinación: lo que nos dice el R cuadrado. Datacamp.
<https://www.datacamp.com/es/tutorial/coefficient-of-determination>

Understanding the Bias-Variance Tradeoff.
(s. f.).

<https://scott.fortmann-roe.com/docs/BiasVariance.html>

Ibm. (2025, 27 febrero). Random Forest.
IBM.

<https://www.ibm.com/mx-es/think/topics/random-forest>

GeeksforGeeks. (2025, 2 septiembre).
Random Forest Hyperparameter Tuning in
Python. GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/random-forest-hyperparameter-tuning-in-python/>

Belcic, I., & Stryker, C. (2025, 20 marzo).
Índice de aprendizaje. IBM.

<https://www.ibm.com/mx-es/think/topics/learning-rate>

Mucci, T. (2025, 20 marzo). Sobreajuste vs.
subajuste. IBM.

<https://www.ibm.com/mx-es/think/topics/overfitting-vs-underfitting>

GeeksforGeeks. (2025a, julio 12). Bias and
Variance in Machine Learning.

GeeksforGeeks.

<https://www.geeksforgeeks.org/machine-learning/bias-vs-variance-in-machine-learning/>